43. Логические операторы: И, ИЛИ, НЕ

На этом уроке мы рассмотрим логические операторы И, ИЛИ и НЕ в языке C++.

Оглавление:

1. Логические операторы
2. Логический оператор НЕ
3. Логический оператор ИЛИ
4. Логический оператор И
5. Короткий цикл вычислений
6. Использование логических операторов И/ИЛИ
7. Законы Де Моргана
8. А где же побитовое исключающее ИЛИ (XOR)?
9. Тест

Логические операторы

В то время как **операторы сравнения** используются для проверки конкретного условия: ложное оно или истинное, они могут проверять только одно условие за определенный промежуток времени. Но бывают ситуации, когда нужно протестировать сразу несколько условий. Например, чтобы узнать, выиграли ли мы в лотерею, нам нужно сравнить все цифры купленного билета с выигрышными. Если в лотерее 6 цифр, то нужно выполнить 6 сравнений, все из которых должны быть true.

Также иногда нам нужно знать, является ли хоть одно из нескольких условий истинным. Например, мы не пойдём сегодня на работу, если больны или слишком устали, или если выиграли в лотерею. 😊 Нам нужно проверить, является ли хоть одно из этих условий истинным. Как это сделать? С помощью логических операторов! Они позволяют проверять сразу несколько условий за раз.

В языке C++ есть 3 логических оператора:

Оператор	Символ	Пример	Операция
Логическое НЕ	`!`	`!x`	true, если x — false и false, если x — true
Логическое И	`&&`	`x && y`	true, если x и y — true, в противном случае — false
Логическое ИЛИ	`||`	`x || y`	true, если x или y — true, в противном случае — false

Логический оператор НЕ

Мы уже сталкивались.

Логический оператор НЕ (!)	
Операнд	**Результат**
true	false
false	true

Если операндом является true, то, после применения логического НЕ, результатом будет false. Если же операнд до применения оператора НЕ был false, то после его применения станет true. Другими словами, логический оператор НЕ меняет результат на противоположный начальному значению. Он часто используется в условных выражениях:

```
bool bIsLarge = (x > 100); // переменная bIsLarge будет true, если x > 100
if (!bIsLarge)
    // доказать что x меньше 100
else
    // доказать что x больше 100
```

Следует помнить, что логический оператор НЕ имеет очень высокий **уровень приоритета**. Новички часто совершают следующую ошибку:

```
#include <iostream>

int main()
{
    int x = 5;
    int y = 7;

    if (!x == y)
        std::cout << "x does not equal y";
    else
        std::cout << "x equals y";

    return 0;
}
```

Результат выполнения программы:

`x equals y`

Но `x` ведь не равно `y`, как это возможно? Поскольку приоритет логического оператора НЕ выше, чем приоритет оператора равенства, то выражение `!x == y` обрабатывается как `(!x) == y`. Так как `x` — это `5`, то `!x` — это `0`. Условие `0 == y` ложное, поэтому выполняется часть else!

Напоминание: Любое ненулевое целое значение в логическом контексте является true. Так как `x = 5`, то `x` вычисляется как true, а вот `!x = false`, т.е. `0`. Использование целых чисел в логических операциях подобным образом может запутать не только пользователя, но и самого разработчика, поэтому так не рекомендуется делать!

Правильный способ написания программы, приведенной выше:

```
#include <iostream>

int main()
{
    int x = 5;
    int y = 7;

    if (!(x == y))
        std::cout << "x does not equal y";
    else
        std::cout << "x equals y";

    return 0;
}
```

Сначала обрабатывается `x == y`, а затем уже оператор НЕ изменяет результат на противоположный.

Правило: Если логический оператор НЕ должен работать с результатами работы других операторов, то другие операторы и их операнды должны находиться в круглых скобках.

Логический оператор ИЛИ

Если хоть одно из двух условий является истинным, то логический оператор ИЛИ является true.

Логический оператор ИЛИ (||)		
Левый операнд	**Правый операнд**	**Результат**
false	false	false
false	true	true
true	false	true
true	true	true

Рассмотрим следующую программу:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int value;
    std::cin >> value;

    if (value == 0 || value == 1)
        std::cout << "You picked 0 or 1" << std::endl;
    else
        std::cout << "You did not pick 0 or 1" << std::endl;

    return 0;
}
```

Здесь мы использовали логический оператор ИЛИ, чтобы проверить, является ли хоть одно из двух условий истинным: левое (`value == 0`) или правое (`value == 1`). Если хоть одно из условий — true или оба сразу true, то выполняться будет стейтмент if. Если же одно из условий не является true, то результат — false и выполняться будет стейтмент else.

Вы можете связать сразу несколько условий:

```
if (value == 0 || value == 1 || value == 2 || value == 3)
    std::cout << "You picked 0, 1, 2, or 3" << std::endl;
```

Новички иногда путают логические ИЛИ (`||`) с побитовым ИЛИ (`|`). Хоть у них и одинаковые названия, функции они выполняют разные.

Логический оператор И

Только при условии, что оба операнда будут истинными, логический оператор И будет true. Если нет, тогда — false.

Логический оператор И (&&)		
Левый операнд	**Правый операнд**	**Результат**
false	false	false
false	true	false
true	false	false
true	true	true

Например, мы хотим узнать, находится ли значение переменной `x` в диапазоне от 10 до 20. Здесь у нас есть два условия: мы должны проверить, является ли `x` больше 10 и является ли `x` меньше 20.

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int value;
    std::cin >> value;

    if (value > 10 && value < 20)
        std::cout << "Your value is between 10 and 20" << std::endl;
    else
        std::cout << "Your value is not between 10 and 20" << std::endl;

    return 0;
}
```

Если оба условия истинны, то выполняется часть if. Если же хоть одно или сразу оба условия ложные, то выполняется часть else.

Как и с логическим ИЛИ, мы можем комбинировать сразу несколько условий И:

```
if (value > 10 && value < 20 && value != 15)
    // доказать что value
else
    // доказать что value не может быть 15
```

Короткий цикл вычислений

Для того, чтобы логическое И возвращало true, оба операнда должны быть истинными. Если первый операнд вычисляется как false, то оператор И должен сразу возвращать false независимо от результата второго операнда (даже без его обработки). Это называют **коротким циклом вычисления** (англ. **«short circuit evaluation»**) и выполняется он, в первую очередь, в целях оптимизации.

Аналогично, если первый операнд логического ИЛИ является true, то и всё условие будет true (даже без обработки второго операнда).

Как и в случае с оператором ИЛИ, новички иногда путают логическое И (`&&`) с побитовым И (`&`).

Использование логических операторов И/ИЛИ

Иногда возникают ситуации, когда смешивания логических операторов И и ИЛИ в одном выражении не избежать. Тогда следует знать о возможных проблемах, которые могут произойти.

Многие программисты думают, что логическое И и ИЛИ имеют одинаковый приоритет (или забывают, что это не так), так же как и сложение/вычитание или умножение/деление. Тем не менее, приоритет логического И выше приоритета ИЛИ. Таким образом, операции с оператором И всегда будут вычислиться первыми (если только операции с ИЛИ не находятся в круглых скобках).

Рассмотрим следующее выражение: `value1 || value2 && value3`. Поскольку приоритет логического И выше, то обрабатываться выражение будет так:

`value1 || (value2 && value3)`.

А не так:

`(value1 || value2) && value3`

Хорошей практикой является использование круглых скобок с операциями. Это предотвратит ошибки приоритета, увеличит читабельность кода и чётко даст понять компилятору, как следует обрабатывать выражения. Например, вместо того, чтобы писать `value1 && value2 || value3 && value4`, лучше записать `(value1 && value2) || (value3 && value4)`.

Законы Де Моргана

Многие программисты совершают ошибку, думая, что `!(x && y)` — это то же самое, что и `!x && !y`. К сожалению, вы не можете использовать логическое НЕ подобным образом.

Законы Де Моргана гласят, что `!(x && y)` эквивалентно `!x || !y`, а `!(x || y)` эквивалентно `!x && !y`.

Другими словами, логические операторы И и ИЛИ меняются местами! В некоторых случаях, это даже полезно, так как улучшает читабельность.

А где же побитовое исключающее ИЛИ (XOR)?

Побитовое исключающее ИЛИ (XOR) — это логический оператор, который используется в некоторых языках программирования для проверки на истинность нечётного количества условий.

Побитовое исключающее ИЛИ (XOR)		
Левый операнд	**Правый операнд**	**Результат**
false	false	false
false	true	true
true	false	true
true	true	false

В языке C++ нет такого оператора. В отличие от логических И/ИЛИ, к XOR не применяется короткий цикл вычислений. Однако его легко можно симулировать, используя оператор неравенства (`!=`):

```
if (x != y)... // x XOR y (true если x или y, но не оба из них)
```

Можно также расширить количество операндов:

```
if (x != y != z != w)... // x XOR y XOR z XOR w (true если нечётное количество из них true)
```

Следует отметить, что вышеприведенные шаблоны XOR работают только, если операнды имеют логический (а не целочисленный) тип данных. Если вы хотите, чтобы это работало и с целыми числами, то используйте **оператор static_cast**.

Форма XOR, которая работает и с другими типами данных (с помощью оператора static_cast мы можем конвертировать любой тип данных в тип bool):

```
if (static_cast<bool>(x) != static_cast<bool>(y))... // x XOR y XOR z XOR w
```

Тест

Какой результат следующих выражений?

Выражение №1: `(true && true) || false`

Выражение №2: `(false && true) || true`

Выражение №3: `(false && true) || false || true`

Выражение №4: `(5 > 6 || 4 > 3) && (7 > 8)`

Выражение №5: `!(7 > 6 || 3 > 4)`

Ответ

Примечание: В ответах объяснение выполняется с помощью стрелочки (=>). Например, `(true || false) => true` означает, что логическое выражение `(true || false)` является `true`.

Выражение №1: `(true && true) || false => true || false => true`

Выражение №2: `(false && true) || true => false || true => true`

Выражение №3: `(false && true) || false || true => false || false || true => false || true => true`

Выражение №4: `(5 > 6 || 4 > 3) && (7 > 8) => (false || true) && false => true && false => false`

Выражение №5: `!(7 > 6 || 3 > 4) => !(true || false) => !true => false`